Week 2 verified agent questionnaire

Name / team:
Repository URL or source ZIP: https://github.com/AimenFH/framework-agent-lab.git
Local model tag and preflight result: `qwen3:8b`, `LIVE PROBE PASSED`

Answer each in two to five clear sentences and refer to your four-minute video. Scripted diagnostics support development; completion requires a live repair.

1. Which part of the Week 1 loop did LangGraph replace? Explain the nodes, message updates, routes, and what the 32-step limit guarantees.

Answer: In Week 1 I wrote the while loop myself: call the model, run the tool it asked for, feed the result back. LangGraph replaces it with three nodes in `build_agent`: `model` adds the system instruction and returns only the new `AIMessage`, `tools` (a `ToolNode`) returns a `ToolMessage` linked to each call, and `verify` decides the result. `MessagesState` appends every update to the history. `tools_condition` sends tool calls to `tools`, which always goes back to `model`, and sends a plain answer to `verify` and then END.

2. Which boundaries do the supplied tools enforce regardless of the model's request? Explain both denials and why a fixed test command still needs review.

Answer: The tools check every path in Python, so the model can only read visible files inside `target-service` (no absolute paths, `..`, hidden folders or symlinks), and reads and writes stop at 12,000 bytes. `edit_pricing` can only touch `orders/pricing.py`, and `run_tests` runs one fixed command after I type exactly `yes`. Reading `../HANDOUT.md` failed because the path leaves the service folder, and replacing `Decimal` with `float` failed because `Decimal` appears several times, so the tool can't tell which one to change.

3. How does your verification node distinguish a final response from verified completion? Explain failed/denied tests and an edit after a passing test. Identify one incomplete result you demonstrated.

Answer: The model's final answer is only text, so my `verify` node ignores it and replays the `ToolMessage` results in order. A successful edit resets the test status, so a pass from before the edit no longer counts. Each later test run replaces the status: `exit_code=0` is a pass, and a failure, `DENIED` or `ERROR` clears it. The run is `verified` only if the last message is a real answer with no tool calls and the latest test result is a pass. In the video I typed no at the approval, the tool returned `DENIED`, and the run ended INCOMPLETE with exit code 2.

4. What evidence shows that the live model read, edited, and tested the service? State the model tag, your regression's expected business result and baseline failure, the final test result, and protected files you checked. What remains unverified?

Answer: The video shows `qwen3:8b` calling `read_file` on the business rules, the acceptance tests and `orders/pricing.py`, then `edit_pricing` once and `run_tests`, each with its tool result printed underneath, and `git diff` shows what the model changed. My regression prices 37.335 × 3 with an 11% discount: discounted goods come to 99.69, so shipping is 5.00 and the total should be EUR 104.69. On the original code it failed with `Decimal('112.005') != Decimal('112.01')`. After the repair, the agent's test run returned `exit_code=0`, my own run passed all 10 tests, and the diff was empty for `validation.py`, `service.py`, `storage.py`, `test_acceptance.py` and `docs`. I haven't verified inputs the tests don't cover, whether another run of the model would also work, or safety without a sandbox.